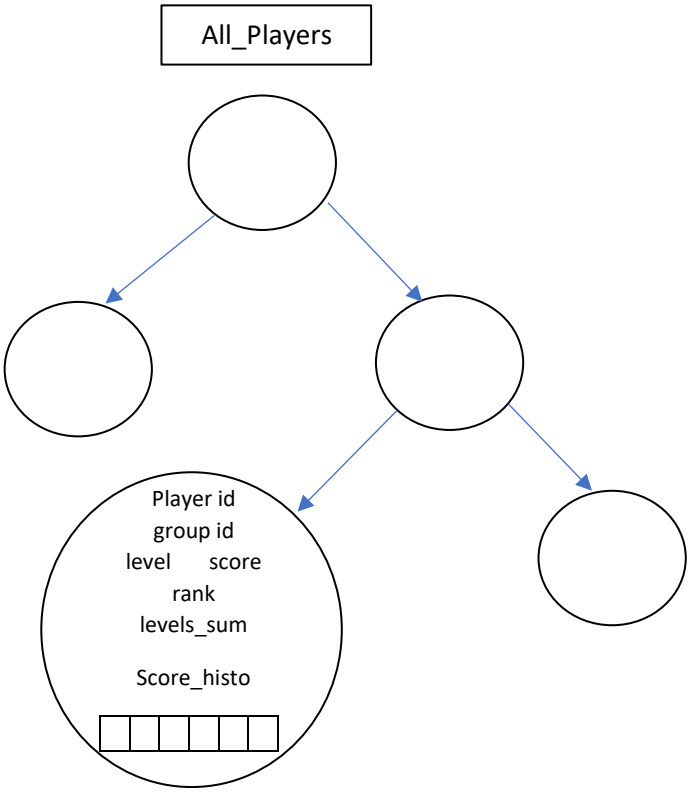
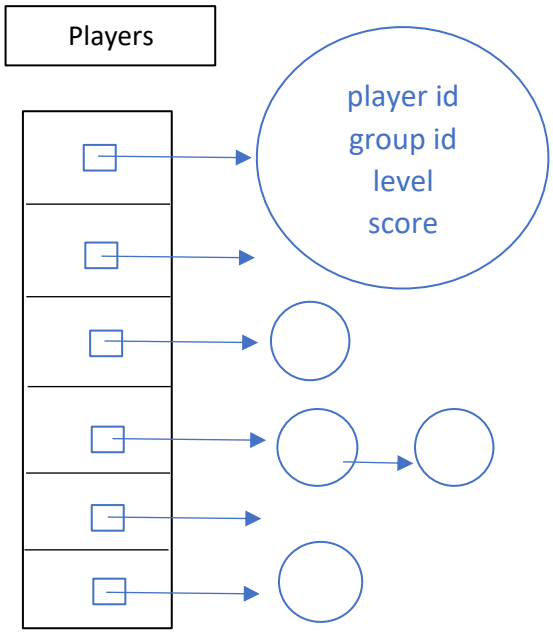
# חלק יבש

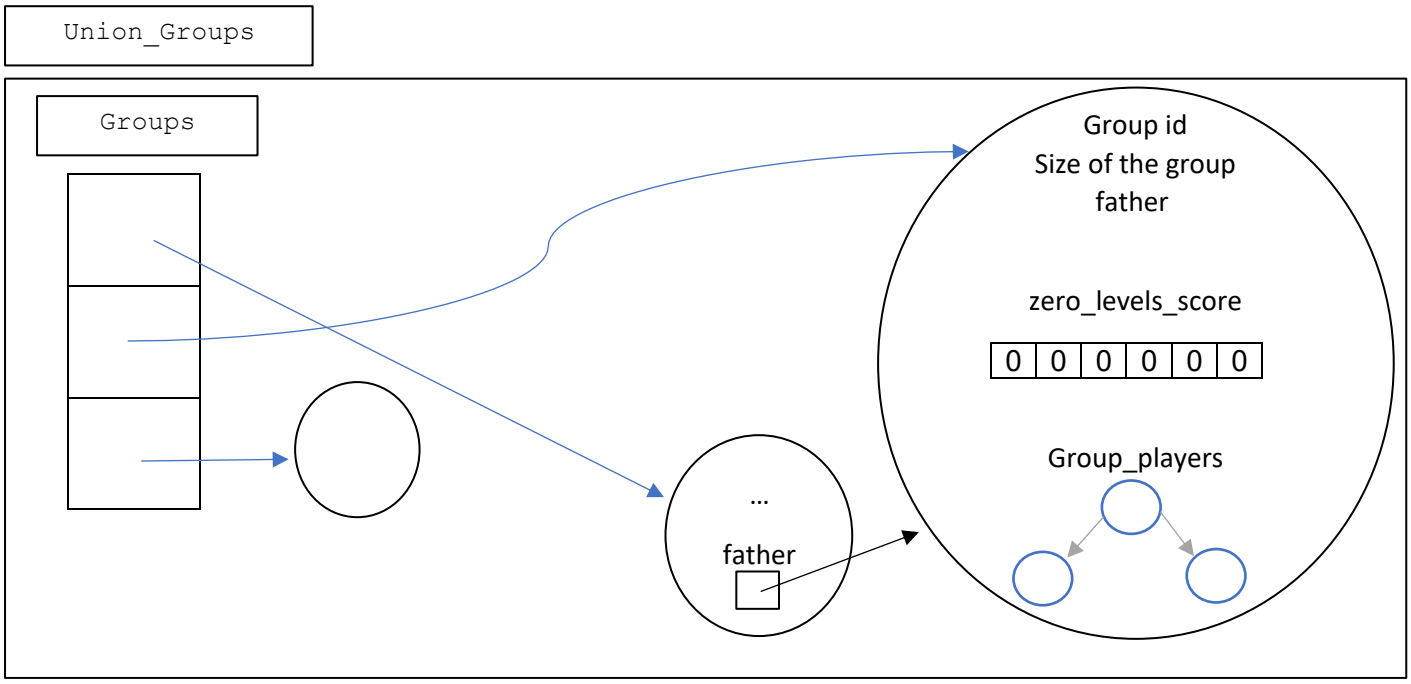
המבנה שלנו :




All_players_zerolevel_scores

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 2 | 0 | 0 |



## תיאור המבנה שלנו :

המבנה מכיל :-

1- "קבוצות זורות (Union Find) בשם "Union Groups" שמכיל את הקבוצות , בכל קבוצה או איחוד של קבוצות קיים עץ דרגות (עם אותם ערכי מידע שיוספרו בנק' 3) של השחקנים השייכים לאותו איחוד קבוצות עם level > 0 , בנוסף להיסטוגרמה בשם "zero_level_score" כך שבמקום ה-i נמצא מספר השחקנים השייכים לאיחוד הקבוצות עם level=0 ו score=i , ומס' הקבוצות באיחוד.
2- טבלת ערבול של כל השחקנים במשחק בשם "players" , השתמשנו ב-chain hashing כמו שראינו בתרגול כדי להתאים בין מס' השחקן למקומו בטבלה – חשוב לציין שהטבלה דינאמית.
3- עץ דרגות AVL של כל השחקנים במשחק שהשלב שלהם > 0 ממויין לפי ה-level של השחקן, בכל חוליה נשמור : 1- מס' השחקן , 2-מס' הקבוצה , 3- שלב השחקן , 4-ה-score של השחקן , 5- מס' הבנים בתת העץ של החוליה (rank) , 6- סכום השלבים בתת העץ של החוליה (levels_sum) , 7- היסטוגרמה בשם "score_histo" כך שבמקום ה-i נמצא מספר השחקנים בתת העץ של החוליה עם score=i.
4- היסטוגרמה בשם "all_players_zerolevel_score" , כך שבמקום ה-i נמצא מספר השחקנים במשחק level=0 ו score=i.

---

## מימוש הפונקציות :

**1-init(int k, int scale) :**

נבנה את המערכת עם טבלת ערבול ריקה בגודל 64, בנוסף לעץ דרגות AVL רק , ונאתחל מבנה קבוצות זרות(Union Find) בו מערך בגודל k כך שבכל מקום במערך מצביע לקבוצה בודדת שקיים בה עץ דרגות AVL רק והיסטוגרמה בגודל 201. בנוסף לכך נבנה היסטוגרמה בגודל 201.

*חישוב סובכיות :

בניית טבלת ערבול ריקה בגודל 64 מתבצעת הסובכיות O(1) , עץ דרגות רק הוא בעצם מצביע ל-NULL ולכן בנייתו גם מתבצעת ב O(1) , ההיסטוגרמה היא בגודל קבוע ששוה ל-201 ולכן בנייתה מתבצעת ב O(1) , לכל קבוצה נבנה עץ רק והיסטוגרמה , ולכן לפי ההסבר הקודם בניית קבוצה אחת תתבצע ב O(1), בUnion Find נבנה k קבוצות ולכן בניית ה Union Find תתבצע ב O(K)

=> סה"כ סובכיות פעולת Init היא O(K) במקרה הגרוע.

---

**2-mergeGroups(void *DS, int GroupID1, int GroupID2) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה .

ניגש לUnion_Groups ונבצע פעולת Union בין איחוד הקבוצות ששייכת אליה קבוצה בעלת מס' GroupID1 , עם איחוד הקבוצות ששייכת אליה קבוצה בעלת מס' GroupID2 , (יתכן שאיחוד הקבוצות מכיל רק את הקבוצה עצמה) . נפרוש את עץ השחקנים השייכים לאיחוד הקבוצות ששיכת אליהם GroupID1 במערך (הפרישה תתבצע בסיור inorder כך שנשמור על הסדר של השחקנים ), כנ"ל עבור האיחוד ששייכת אליו GroupID2 . נמזג את שני המערכים למערך אחד גדול , נבנה עץ כמעט שלם רק בגודל מס' השחקנים של שתי הקבוצות , נבצע סיור inorder על העץ הרק החדש , בכל חוליה נוסיף את השחקן הראשון במערך הממוזג , נמחק אותה מהמערך ונעבור חוליה וכ"ו, ובכך מזגנו שני העצים . נשנה את עץ השחקנים של ראש איחוד הקבוצות (שעכשיו שייכות אליהם GroupID1 ו GroupID2) להצביע לעץ השחקנים החדש.

***חישוב סובכיות :**

פעולת union בה מבצעים פעולת find כדי לגשת לראש איחוד הקבוצות שהקבוצה שייכת אליו ולכן סובכיות משוערכת היא O(log*(k)) , פרישת עצי הקבוצות במערך בעזרת סיור inorder ומיזוגם מתבצע ב O(n) . ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר הקבוצות (k) , מתקיים שהפונקציה היחידה שתלויה רק במס' השחקנים היא פונקצית המיזוג ולכן ולכן fn=O(n) ולכן הסובכיות המשוערכת של פעולת merge היא O(log*(k)+n) משוערך.

---

**3- addPlayer(void *DS, int PlayerID, int GroupID, int score) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

נבנה שחקן חדש בעל מס שחקן PlayerID , השייך לקבוצה מס' GroupID עם level=0 ו score.=score

נחשב את מקומו המתאים בטבלת הערבול לפי המספר שלו מודלו גודל המערך, ונוסיף את השחקן לרשימה במקום הזה . במקרה של חריגה מפאקטור העומס שהגדרנו אותו בטבלת הערבול להיות 2 , נגדיל את הטבלה פי 2 , ונעביר את כל השחקנים מהטבלה הישנה לחדשה , כך שלכל שחקן נחשב את מקומו המתאים החדש בטבלה, ובסוף נמחק את החבלה הישנה.

ניגש להיסטוגרמה "all_players_zerolevel_score" למקום ה-score , ונגדיל אותו ב1.

נמצא את ראש איחוד הקשבוצות השייכת אליו GroupID וניגש להיסטוגרמה שלו "zero_level_score" למקום ה-score , ונגדיל אותו ב1.

***חישוב סובכיות :**

מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של O(log*(k)) , גישה למערך ועדכון ערכי ההיסטוגרמות מתבצע ב O(1) , ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה, מתקיים שהסובכיות המשוערכת של פעולת addPlayer היא O(log*(k)) משוערך.

---

**4- removePlayer(void *DS, int PlayerID) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

נמצא את השחקן בטבלת הערבול לפי המס' שלו , אם לא נמצא נחזיר שגיאה .

נמצא את ראש איחוד הקבוצות שאליה שייך השחקן , (לפי מס' הקבוצה הראשום אצל השחקן שמצאנו בשלב הקודם) נקרא לו group_head.

אם השחקן בעל level=0 : ניגש להיסטוגרמה "all_players_zerolevel_score" למקום ה-score , ונקטין אותו ב1. וניגש להיסטוגרמה "zero_level_score" של group_head למקום ה-score , ונקטין אותו ב1.

אחרת , level>0 : נמחק את השחקן מעץ all_players וניגש לעץ group_players של-group_head ונמחק אותו משם.

נמחק את השחקן מטבלת הערבול , במקרה ומס' השחקנים אחרי המחיקה קטן מחצי גודל הטבלה , נקצה טבלת ערבול בגודל שווה לחצי גודל הטבלה הישנה , ונעביר את כל השחקנים מהטבלה הישנה לחדשה , כך שלכל שחקן נחשב את מקומו המתאים החדש בטבלה, ובסוף נמחק את החבלה הישנה.

***חישוב סובכיות :**

מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של O(log*(k)) , מציאת השחקן בטבלת הערבול מתבצע בO(1) בממוצע , מחיקת השחקן מהעץ בO(log(n)) , ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר השחקנים (n), מתקיים שהפונקציה היחידה שתלויה רק במס' הקבוצות היא פונקצית find ולכן f(k)=O(log*(k)) ולכן הסובכיות המשוערכת של פעולת removePlayer היא O(log*(k)+log(n)) משוערך.

---

**5- increasePlayerIDLevel(void *DS, int PlayerID, int LevelIncrease) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

נמצא את השחקן בטבלת הערבול , אם לא נמצא נחזיר שגיאה .

נוסיף לשחקן שמצאנו (שנמצא בטבלת הערבול) LevelIncrease ל level שלו.

נמצא את ראש איחוד הקבוצות שאליה שייך השחקן , (לפי מס' הקבוצה הרשום אצל השחקן שמצאנו בשלב הקודם) נקרא לו group_head.

אם השחקן לפני הוספת LevelIncrease היה בעל level=0 : ניגש להיסטוגרמה"all_players_zerolevel_score" למקום ה-score , ונקטין אותו ב1. וניגש להיסטוגרמה "zero_level_score" של group_head למקום ה-score , ונקטין אותו ב1. נכניס לעץ all_players , שחקן בעל מס' PlayerID , השייך לקבוצה בעלת אותו מס' של קבוצת השחקן שמצאנו בטבלת הערבול עם level= LevelIncrease ואתו score של השחקן שמצאנו בטבלת הערבול . נבצע אותה הכנסה לעץ group_players ששייך ל-group_head .

אחרת (לפני הוספת LevelIncrease היה בעל level>0) : נמחק את השחקן בעל מס' PlayerID מעץ all_players , ו נכניס לאותו עץ( all_players) , שחקן בעל מס' PlayerID , השייך לקבוצה בעלת אותו מס' של קבוצת השחקן שמצאנו בטבלת הערבול עם level= level_after_increasing ואתו score של השחקן שמצאנו בטבלת הערבול . נבצע אותה מחיקה ואחריה הכנסה לעץ group_players ששייך ל-group_head .

*level_after_increasing = old_level + LevelIncrease = השלב אחרי ההוספה.

***חישוב סובכיות :**

מציאת השחקן בטבלת הערבול מתבצע ב(O(1 בממוצע , עדכון שדה ה-level שלו מתבצע ב (O(1 , מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של ((O(log*(k , גישה למערך ועדכון ערכי ההיסטוגרמות(במקרה והיה בשלב אפס) מתבצע ב (O(1 , מחיקת השחקן מהעץ והוספתי לעץ מתבצע ב ((O(log(n , ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר השחקנים (n), מתקיים שהפונקציה היחידה שתלויה רק במס' הקבוצות היא פונקצית find ולכן f(k)=O(log*(k)) ולכן הסובכיות המשוערכת של פעולת increasePlayerLevel היא ((O(log*(k)+log(n משוערך.

---

**6-changePlayerIDScore(void *DS, int PlayerID, int NewScore) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

נמצא את השחקן בטבלת הערבול , אם לא נמצא נחזיר שגיאה .

נעדכן לשחקן שמצאנו (שנמצא בטבלת הערבול) את שדה ה-score להיות NewScore.

נמצא את ראש איחוד הקבוצות שאליה שייך השחקן , (לפי מס' הקבוצה הרשום אצל השחקן שמצאנו בשלב הקודם) נקרא לו group_head.

אם השחקן בעל level=0 : ניגש להיסטוגרמה "all_players_zerolevel_score" למקום ה-score (הישן שהיה לשחקן לפני עדכון השדה) ונקטין אותו ב1, ונגדיל את המקום ה- NewScore(באותה הסיסטוגרמה) ב1 .וניגש להיסטוגרמה "zero_level_score" של group_head למקום ה-score (הישן שהיה לשחקן לפני עדכון השדה) ונקטין אותו ב1. , ונגדיל את המקום ה- NewScore (באותה הסיסטוגרמה) ב1 .

אחרת (השחקן בעל level>0) , נחפש את השחקן בעץ all_players ונעדכן את שדה ה-score שלו להיות NewScore , וניגש למידע score_histo של החוליה בעץ (שהוא בעצם היסטוגרמה שמכילה במקום ה-i את מס' השחקנים בעלי score=i בתת העץ של החוליה – כולל החוליה עצמה) ונקטין 1 במקום ה-score (הישן שהיה לשחקן לפני עדכון השדה) ונוסיף 1 למקום ה-NewScore , ונבצע את החיסור וההוספה במקומות score ו- NewScore במידע score_histo לאורך כל המסלול מהחוליה עד שורש העץ. כנ"ל נעשה בעץ group_players השייך ל-group_head .

## לגבי הבונוס:

נחסום את מספר האיברים הממופים לאותה חוליה ( השייכים לאותה רשימה) ע"י מספר קבוע כלשהו, נגיד 20. נבצע את זה ע"י שמירת מונה הסופר כמות האיברים השייכים לכל רשימה ואם מונה זה עובר את הקבוע שבחרנו, אנחנו נעשה expand_hashtable כרגיל.

כעת חיפוש של איבר כלשהו ב hash table יעלה לנו **(1)o במקרה הגרוע**. המשך ניתוח סיבוכיות הזמן של הפעולה דומה לסעיף הקודם 1 על 1.

**חישוב סובכיות :**

מציאת השחקן בטבלת הערבול מתבצע ב(O(1 בממוצע , עדכון שדה ה-level שלו מתבצע ב (O(1 , מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של ((O(log*(k , גישה למערך ועדכון ערכי ההיסטוגרמות מתבצע ב (O(1 , מציאת השחקן בעץ ועדכון שדה score שלו מתבצע ב ((O(log(n, ולפי ההנחיות לגבי

הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר השחקנים (n), מתקיים שהפונקציה היחידה שתלויה רק במס' הקבוצות היא פונקצית find ולכן f(k)=O(log*(k)) ולכן הסובכיות המשוערכת של פעולת changePlayerIDScore היא O(log*(k)+log(n)) משוערך.

---

**-7 int getPercentOfPlayersWithScoreInBounds (void *DS, int GroupID, int score, int lowerLevel, higherLevel, double * players)**

הערות : 1-עץ הדרגות ממויין לפי שלב השחקן ובמקרה של שיוויון שלבים לפי player_id. 2-בכל חוליה בעץ קיים שדה rankשמור מס' הבנים בתת העץ של החוליה . 3- בכל חוליה בעץ קיים שדה levels_sum ששומר סכום הlevels של הבנים בתת העץ של החוליה . 4- בכל חוליה בעץ קיימת היסטוגרמה score_histo בהבמקום ה-i שומרים את מס' הבנים שיש להם score=i בתת העץ של החוליה.

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

במקרה ש-GroupID>0 :

נמצא את ראש איחוד הקבוצות שאליה שייכת הקבוצה, (לפי GroupID) נקרא לו group_head.

אם higherLevel<0 : נחזיר 0 כי אין שחקנים בשלב שלילי.

אם higherLevel=0 : ניגש להיסטוגרמה zero_levels_score של group_head (שם נמצאים כל השחקנים השייכים לקבוצה בעלי level=0 , נסכום את איברי ההסטוגרמה(סריקה על המערך וסכימה ערכי כל תא) ,זהו מס' השחקנים השייכים לקבוצה ונמצאים ברמה שהינה בטווח [lowerLevel, higherLevel] נסמנו players_num, ניגש להיסטוגרמה במקום score(שם נמצא מס' השחקנים בשלב 0 עם player_score=score נסמנו players_with_score, אם players_num=0 זאת אומרת שאין שחקנים בטווח הדרוש ולכן נחזיר שגיאה, אחרת , נבצע:players=(double)players_with_score/players_num* ונחזיר הצלחה.

אחרת (higherLevel>0), ניגש לעץ group_players של group_head , נחשב: (1)את מס' השחקנים שמקיימים player_level<=higherLevel, (2)ואת מס' השחקנים שמקיימים player_level<=higherLevel וגם player_score=score, (3)ואת מס' השחקנים שמקיימים player_level<lower_level, (4)ואת מס' השחקנים שמקיימים player_level<lower_level וגם player_score=score (כל החישובים נעשים לפי אלגוריתם (*)) . נחסר את (3) מ-(1) ונקבל את מס' השחקנים בעץ השייכים לקבוצה ונמצאים ברמה שהינה בטווח [lowerLevel , higherLevel], נסמנו players_num , ונחסר (4) מ-(2)ונקבל את מס' השחקנים בעץ השייכים לקבוצה ונמצאים ברמה שהינה בטווח [lowerLevel , higherLevel] וגם מקיימים player_score=score נסמנו players_with_score.

אם lowerLevel>0 , זאת אומרת שכל השחקנים שבטווח נמצאים בעץ ולכן כבר חשבנו אותם ואז אם players_num=0 זאת אומרת שאין שחקנים בטווח הדרוש ולכן נחזיר שגיאה, אחרת , נבצע:players=(double)players_with_score/players_num* ונחזיר הצלחה.

אחרת (lowerLevel<=0) , אז קיימים עוד שחקנים שהשלב שלהם 0 ולכן לא נמצאים בעץ וכן נמצאים בהיסטוגרמה , נסכום את איברי ההיסטוגרמה ונוסיף את התוצאה ל players_num , ונוסיף לplayers_with_score את ערך המקום score בהיסטוגרמה , ואז אם players_num=0 זאת אומרת שאין שחקנים בטווח הדרוש ולכן נחזיר שגיאה, אחרת , נבצע:players=(double)players_with_score/players_num* ונחזיר הצלחה.

אחרת (GroupID=0) : נבצע בדיוק את התהליך למעלה רק במקום העץ group_players של group_head , נשתמש בעץ all_players , ובמקום ההיסטוגרמה zero_levels_score של group_head נשתמש בהיסטוגרמה all_players_zerolevel_scores באופן סמטרי.

אלגוריתם (*): אלגוריתם המבקל level ומס' שחקן , ומוצא את מס' השחקנים בעלי player_level<=level ו player_id<id :

נגדיר שני משתנים : players_num=0 , players_with_score=0 .

נגדיר שחקן בעל player_level=level ו player_id=id ונכניס אותו לעץ , נסמנו temp.

יהי path מסלול החיפוש של השחקן temp , לכל צומת player שקטן מ-temp ב-path , נבצע players_num=1+player->left->rank , players_with_score = player->left->score_histo[score] , ואם player->score=score נבצע עוד players_with_score = players_with_score+1 ונתקדם ימינה . אחרת(הצומת player>temp) מתקדם שמאלה.

בסוף נמחק את temp מהעץ.

כדי לחשב את (1) ו (2) נריץ את האלגוריתם עם level=higherLevel ו id=∞.

כדי לחשב את (3) ו (4) נריץ את האלגוריתם עם level=lowerLevel ו id = -∞.

סובכיות ביצוע האלגוריתם היא כעומק העץ ולכם O(log(n)).

***חישוב סובכיות :**

מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של O(log*(k)) **,** חישוב מס' השחקנים בעלי level=0 וכמה מהם בעלי score=score ב(O(1 (גודל ההיסטוגרמה קבוע) . ביצוע האלגוריתם ב(O(log(n)) , חישוב את האחוז ב O(1) , . ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר הקבוצות (k) , מתקיים שהפונקציה היחידה שתלויה רק במס' השחקנים היא פונקצית ביצוע האלגוריתם ולכן f(n)=O(log(n)) ולכן הסובכיות המשוערכת של פעולת getPercentOfPlayersWithScoreInBounds היא O(log*(k)+log(n)) משוערך.

---

**8- averageHighestPlayerLevelByGroup(void *DS, int GroupID, int m, double * avgLevel) :**

נבדוק תקינות הקלט , במקרה של אי תקינות נחזיר שגיאה.

נגדיר players_levels_sum=0 .

במקרה ש-GroupID>0 :

נמצא את ראש איחוד הקבוצות שאליה שייכת הקבוצה, (לפי GroupID) נקרא לו group_head.

נסמן את מס' השחקנים בעץ group_players של group_head ב-players_in_tree(נתון בשדות העץ) – זהו מס' השחקנים השייכים לקבוצה בעלי level>0, ואת מס' השחקנים בהיסטגראמה ב players_with_zeroLevel – זהו מס' השחקנים השייכים לקבוצה בעלי level=0.

אם m>players_in_tree+players_with_zeroLevel נחזיר שגיאה .

אם 0 = players_in_tree+players_with_zeroLevel , נחזיר 0 כי אין שחקנים בקבוצה.

אם m>=players_in_tree ניגש לשורש העץ ,ונבצע players_levels_sum = root->levels_sum(זהו שדה בחוליה ששומר את סכום ה-levels בתת העץ של החוליה , בגלל שאנחנו בשורש זהו סכום ה levels של של השחקנים) , ואז נבצע avgLevels = (double)players_levels_sum / m* ונחזיר הצלחה.

אחרת , m<players_in_tree , נמצא את השחקן בעל האנדקס n-m+1 בעזרת אלגוריתם select שנלמד בתרגול – כך ש n הוא מס' השחקנים בעץ , נסמנו temp. נחשב את סכום ה-levels של השחקנים player כך ש player<=temp בעזרת השדה levels_sum ששומר את סכום ה-levels בתת העץ של החוליה , בשיתה שארינו בהרצאה של "מציאת סכום האיברים הקטנים מאיבר x" , בשורש העץ שמור סכום כל ה-levels של השחקנים , ולכן נחסר ממנו את סכום ה-levels של הplayers כך שplayer<=temp ונוסיף לtemp->level (זהו סכום m האיברים ברמות הגבוהות ביותר נכניס אותו ל-players_levels_sum ) , ואז נבצע avgLevels = * (double)players_levels_sum / m ונחזיר הצלחה.

נבצע:players_with_score/players_num(double)=players* ונחזיר הצלחה.

אחרת (GroupID=0) : נבצע בדיוק את התהליך למעלה רק במקום העץ group_players של group_head , נשתמש בעץ all_players , ובמקום ההיסטוגרמה zero_levels_score של group_head נשתמש בהיסטוגרמה all_players_zerolevel_scores באופן סמטרי.

***חישוב סובכיות :**

מציאת ראש איחוד הקבוצות אליה שייכת GroupID בעזרת פעולת find מתבצע בזמן משוערך של O(log*(k)) , חישוב סכום השלבים של m השחקנים שבשלבים הכי גבוהים מתבצע לפי האלגוריתם שראינו בהרצאה ומתבצע בO(log(n)) . ולפי ההנחיות לגבי הסובכיות המשוערכת בסוף קובץ העבודה הנוגעת לחלק שנובע ממספר הקבוצות (k) , מתקיים שהפונקציה היחידה שתלויה רק במס' השחקנים היא פונקצית חישוב סכום השלבים ולכן f(n)=O(log(n)) ולכן הסובכיות המשוערכת של פעולת averageHighestLevelInGroup היא O(log*(k)+log(n)) משוערך.

---

**9-Quit(void **DS) :**

נמחק את ההיסטוגרמה all_players_zerolevel_score .

נעבור על עץ הדרגות all_players ונמחק את כל הצמתים שלו , בסוף נמחק את העץ.

ניגש לUnion Groups , נעבור על המערך המכיל את הקבוצות ונמחק כל קבוצה ובסוף נמחק את המערך.

נעבור על טבלת הערבול ונמחק ממנה את כל השחקנים , ובסוף נמחק אותה

***חישוב סובכיות :**

ההיסטוגרמה היא בגודל קבוע ולכן מחיקת ההיסטוגרמה מתבצעת בO(1) , העץ all_players מכיל את כל השחקנים בעלי level>0 ולכן לכל היותר מכיל n צמתים , בכל חוליה נמחק את ההסטוגרמה שהיא בגודל קבוע ולכן מחיקת כל חוליה תתבצע ב O(1) ומחיקת העץ כולו תתבצע במקרה הגרוע ב O(n) .

ניגש לUnion Group , נעבור על מערך הקבוצות , בכל קבוצה : רק אם היא ראש של איחוד נמחק את עץ השחקנים השייכים לאיחוד , ונמחק את הקבוצה(שבה רק מוחקים את ההיסטוגרמה) , ולכן נמחק K קבוצות , מחיקת כל קבוצה שאינה ראש איחוד בO(1) , ומחיקת קבוצה שהיא ראש איחוד מתבצע בO(p)כך ש-p הוא מס' השחקנים השייכים לאיחוד , ולכן מחיקת של ראשי האיחודים מתבצע בO של מס' השחקנים במשחק + מס' ראשי איחודים , וכאן מחיקת Union Group מתבצע בO(K+n).

נעבור על טבלת הערבול ונמחק את כל השחקנים ממנה ובסוף נמחק אותה , מחיקתה תתבצע בO(n).

=> ולכן סה"כ פעולת Quit תתבצע בO(K+n) במקרה הגרוע.

---

**סובכיות מקום :** ההסטוגרמה היא בגודל קבוע ולכן בוסכיות המקום שלה היא O(1).

עץ all_players מכיל את כל השחקנים בעלי level>0 ולכן לכל היותר מכיל n צמתים ולכן בוסכיות המקום שלו היא O(n) במקרה הגרוע.

טבלת הערבול מכילה כל השחקנים ולכן סובכיות המקום שלה היא O(n).

Union Group מכיל מערך בגודל K , כל האיברים במערך(שהם K קבוצות) ביחד מכילים K היסטוגרמות בגודל קבוע ולכן סובכיות המקום שלהן היא O(K) , בנוסף לכך כל האיברים ביחד מכילים לכל היותר כל השחקנים בעצי השחקנים של איחודי הקבוצות , ולכן סובכיות המקום של כל עצי השחקנים הוא O(n) , ומכאן סובכיות במקום של Union Group היא לכל היותר O(K+n).

=>ומכאן סה"כ סובכיות מקום של המבנה שלנו היא O(K+n) במקרה הגרוע.